

NASM 32-bit ASSEMBLY STUDENT

EXERCISE SET

Module 1: Getting Started with NASM 32-bit

5 Programming Exercises | Test Cases | Guide Questions | Coding Space

Student Name	Francis Karl G Yulo
Section	
Date	
Instructor	

Module Focus

NASM program structure, .text and .data sections, global _start, sys_write, sys_exit, multiple output calls, exit status, and basic debugging.

Workbook Goal

Solve five new problems that use the same core concepts as the module but use different scenarios and data. Build each program with NASM 32-bit and Linux int 0x80 system calls.

Standard Build Commands

```
nasm -f elf32
exercise.asm -o
exercise.o ld -m elf_i386
-o exercise exercise.o
./exercise
```

1. Assembly Lab Welcome Board

Module 1 Exercise

Problem Statement. Create a NASM 32-bit program that prints a three-line welcome board for an assembly laboratory. Use separate string labels and one sys_write call for each line, then exit cleanly with status 0.

Required Input	No keyboard input. Three initialized strings in .data.
Required Output	Three lines: "Assembly Laboratory", "Mode: NASM 32-bit", and "Status: Ready".

Test Cases

Test Case	Sample Input / Change	Expected Result / Output
1	Use the default strings.	Assembly Laboratory Mode: NASM 32-bit Status: Ready
2	Change the second line to "Mode: Linux ELF32".	Only the second output line changes; the other lines remain the same.
3	Add an exclamation mark to the third line.	The full edited third line prints because its length is computed from the label.

Questions

1. What is the purpose of global _start?
used as a primary entry point
2. Which four registers are prepared before each sys_write call?
eax,ebx,ecx and edx
3. Why must EAX be loaded again with SYS_WRITE before every separate write?
because the kernel overwrites the EAX register with the return value of the system call once it finishes executing

NASM Code / Work Area

Write your complete NASM program below. Use the test cases to verify your result.

```
SYS_EXIT equ 1
```

```
SYS_WRITE equ 4
```

```
STDOUT equ 1
```

```
section .text
```

```
global _start
```

```
_start:
```

```
mov eax, SYS_WRITE
```

```
mov ebx, STDOUT
```

```
mov ecx, place
```

```
mov edx, len_place
```

```
int 0x80
```

```
mov eax, SYS_WRITE
```

```
mov ebx, STDOUT
```

```
mov ecx, mode
```

```
mov edx, len_mode
```

```
int 0x80
```

```
mov eax, SYS_WRITE
```

```
mov ebx, STDOUT
```

```
mov ecx, status
```

```
mov edx, len_status
```

```
int 0x80
```

```
mov eax, SYS_EXIT
```

```
xor ebx, ebx
```

```
int 0x80
```

```
section .data
```

```
place db 'Assembly Laboratory', 0xa
```

```
len_place equ $ - place
```

```
mode db 'Mode: Linux ELF32', 0xa
```

```
len_mode equ $ - mode
```

```
status db 'Status: Ready!', 0xa
```

```
len_status equ $ - status
```

2. Toolchain Status Screen

Module 1 Exercise

Problem Statement. Print three independent build-status messages: Assembler ready, Linker ready, and Program ready. Each message must be stored separately and printed with its own `sys_write` call.

Required Input	No keyboard input.
Required Output	Assembler ready Linker ready Program ready

Test Cases

Test Case	Sample Input / Change	Expected Result / Output
1	Default source.	Assembler ready Linker ready Program ready
2	Remove 0xa from the first message.	The first and second messages run together on one line.
3	Temporarily skip the second <code>sys_write</code> block.	Only the first and third messages appear.

Questions

4. What does 0xa represent in the data section?

represent hexadecimal 10

5. What does EDX control during `sys_write`?

number of bytes

6. What will happen if ECX points to the wrong string label?

runtime errors

NASM Code / Work Area

Write your complete NASM program below. Use the test cases to verify your result.

```
``asm

SYS_EXIT equ 1

SYS_WRITE equ 4

STDOUT equ 1

section .text

global _start

_start:

; Print Assembler status

mov eax, SYS_WRITE

mov ebx, STDOUT

mov ecx, assembler
```

```
mov edx, len_assembler
```

```
int 0x80
```

```
; Print Linker status
```

```
mov eax, SYS_WRITE
```

```
mov ebx, STDOUT
```

```
mov ecx, linker
```

```
mov edx, len_linker
```

```
int 0x80
```

```
; Print Program status
```

```
mov eax, SYS_WRITE
```

```
mov ebx, STDOUT
```

```
mov ecx, program
```

```
mov edx, len_program
```

```
int 0x80
```

```
; Exit
```

```
mov eax, SYS_EXIT
```

```
xor ebx, ebx
```

```
int 0x80
```

```
section .data
```

```
assembler db 'Assembler ready!', 0xa
```

```
len_assembler equ $ - assembler
```

```
linker db 'Linker ready!', 0xa
```

```
len_linker equ $ - linker
```

```
program db 'Program ready!', 0xa
```

```
len_program equ $ - program
```

3. Exit Status Reporter

Module 1 Exercise

Problem Statement. Write a program that prints "Task complete." and then returns exit status 25 to the shell. After running the program, use echo \$? to verify the status.

Required Input	No keyboard input. The shell command echo \$? is used after execution.
Required Output	Task complete. followed by shell exit status 25.

Test Cases

Test Case	Sample Input / Change	Expected Result / Output
1	Run the default program, then echo \$?.	Task complete. 25
2	Change the exit value from 25 to 7.	The message is unchanged; echo \$? displays 7.
3	Change the exit value to 0.	The message is unchanged; echo \$? displays 0.

Questions

- Which register carries the exit status for sys_exit?
the number carries on to eax
- Why can a program print correct output but still return a nonzero exit status?
because they are 2 different variables
- What shell variable is displayed by echo \$?
exit status of the most recently executed foreground command

NASM Code / Work Area

Write your complete NASM program below. Use the test cases to verify your result.

```
SYS_EXIT equ 1  
  
SYS_WRITE equ 4  
  
STDOUT equ 1
```

```
section .data
```

```
message db 'Task Complete', 10
```

```
message_len equ $ - message
```

```
section .text
```

```
global _start
```

```
_start:
```

```
    mov eax, SYS_WRITE
```

```
    mov ebx, STDOUT
```

```
    mov ecx, message
```

```
    mov edx, message_len
```

```
    int 0x80
```

```
    mov eax, SYS_EXIT
```

```
mov ebx, 25
```

```
int 0x80
```

4. Three-Step Workflow Block

Module 1 Exercise

Problem Statement. Store three workflow lines inside one contiguous buffer and print the whole block with a single `sys_write` call. The lines are Step 1: Edit, Step 2: Assemble, and Step 3: Run.

Required Input	No keyboard input. One multi-line buffer in <code>.data</code> .
Required Output	Step 1: Edit Step 2: Assemble Step 3: Run

Test Cases

Test Case	Sample Input / Change	Expected Result / Output
1	Default buffer.	Step 1: Edit Step 2: Assemble Step 3: Run
2	Add "Step 4: Debug" to the same buffer.	The fourth line appears without adding another <code>sys_write</code> call.
3	Set EDX temporarily to 12.	Only the first 12 bytes of the block are printed.

Questions

10. How can one label represent several printed lines?

represents the entire block message

11. Why does one `sys_write` work for the entire block?

because it tells the kernel the exact size of the block

12. What determines the exact number of bytes printed?

edx

NASM Code / Work Area

Write your complete NASM program below. Use the test cases to verify your result.

```
SYS_EXIT equ 1

SYS_WRITE equ 4

STDOUT equ 1


section .data


message db 'Step 1:Edit', 10
        db 'Step 2:Assemble', 10
        db 'Step 3:Run', 10
        db 'Step 4:test',10


message_len equ $ - message


section .text


global _start
```

```
_start:

mov eax, SYS_WRITE

mov ebx, STDOUT

mov ecx, message

mov edx, 12

int 0x80


mov eax, SYS_EXIT

mov ebx, 0

int 0x80
```

5. Repair the System Notice

Module 1 Exercise

Problem Statement. Create the corrected version of a broken notice program. The faulty version omits global `_start`, places the message address in `EBX` instead of `ECX`, uses `SYS_READ` instead of `SYS_WRITE`, and exits without loading `SYS_EXIT`. Repair all four problems so the notice prints correctly and the program exits with status 0.

Required Input	No keyboard input. Repair the four stated logical errors.
----------------	---

Required Output	System notice: READY
-----------------	----------------------

Test Cases

Test Case	Sample Input / Change	Expected Result / Output
1	Correct all four errors.	System notice: READY
2	Put the message address in EBX again.	The sys_write arguments are wrong and the expected notice is not printed correctly.
3	Use the correct code but change the message text.	The edited notice prints when the length constant is also computed from the label.

Questions

13. Why must the buffer address be placed in ECX for sys_write?

Because we are using linux for this

14. What is the role of global _start during linking?

it is supposed to be for primary entry point

15. Why are logical register mistakes not always detected by the assembler?

because it only detects syntax because logical are human made errors

NASM Code / Work Area

Write your complete NASM program below. Use the test cases to verify your result.

```
SYS_EXIT equ 1
```

```
SYS_WRITE equ 4
```

```
STDOUT equ 1
```

```
section .data
```

```
notice db 'System notice: READY', 10
```

```
notice_len equ $ - notice
```

```
section .text
```

```
global _start
```

```
_start:
```

```
    mov eax, SYS_WRITE
```

```
    mov ebx, STDOUT
```

```
    mov ecx, notice
```

```
    mov edx, 50./repair_system_status
```

```
    int 0x80
```

```
    mov eax, SYS_EXIT
```

```
    mov ebx, 0
```

```
    int 0x80
```

